

IFNMG

Minicurso Design Patterns

Orientação a objetos, SOLID e padrões em TypeScript

AFRANIO CAIRES

2026

Sumário

Apresentação	3
Orientação a objetos	4
Objeto e classe	4
Estado e comportamento	4
Encapsulamento	4
Abstração	5
Interface	5
Classe abstrata	5
Herança e composição	6
Polimorfismo	6
Acoplamento e coesão	7
SOLID	7
S: Responsabilidade Única	7
O: Aberto/Fechado	8
L: Substituição de Liskov	8
I: Segregação de Interfaces	9
D: Inversão de Dependência	9
DIP, IoC, DI e container.....	10
Inversão de Controle (IoC)	10
Injeção de Dependência (DI)	10
Container de DI	10
Tempo de vida das dependências	11
Design Patterns	12
Categorias clássicas	12
Strategy	12
Factory Method	12
Observer	13
Adapter	13
Decorator	14
Como escolher um pattern	15
Quando não usar	15
Prática: preparando o ambiente.....	16
Baixando o repositório	16
Conhecendo o projeto	16
Prática: refatoração passo a passo	17
Passo 0: ler o código e listar os motivos para mudar	17
Passo 1: Strategy para o desconto	17

Passo 2: Factory para escolher o desconto	18
Passo 3: Strategy e Factory para o pagamento	19
Passo 4: Adapter para o cartão	21
Passo 5: Observer para as ações depois de finalizar	22
Passo 6: montar tudo no index.ts	24
Como o projeto ficou	25
Exercícios	26
Para continuar estudando	26
Fechamento.	26



Apresentação

Esta apostila acompanha o primeiro dia do minicurso de Design Patterns do Instituto Federal do Norte de Minas Gerais. A primeira parte cobre a teoria, com exemplos curtos em TypeScript. A segunda parte é prática: você vai baixar o projeto de uma cafeteria e refatorá-lo aplicando, um de cada vez, os padrões estudados.

A aula segue uma ordem: primeiro a base, depois os princípios e, por fim, os padrões.

1. Orientação a objetos com exemplos de código.
2. Problemas de design: acoplamento, coesão e mudança.
3. SOLID, entendendo o que cada princípio tenta evitar.
4. DIP, IoC, DI e container, na ordem em que as ideias se apoiam.
5. Design Patterns: Strategy, Factory Method, Observer, Adapter e Decorator.
6. Como reconhecer quando um pattern ajuda e quando só atrapalha.
7. Prática guiada no projeto da cafeteria.

O que você precisa para a prática

- Node.js 20 ou mais recente e npm;
- Git;
- um editor de código com suporte a TypeScript, como o VS Code.



Orientação a objetos

Os padrões de projeto são escritos com o vocabulário da orientação a objetos. Antes de chegar neles, vale revisar os conceitos que vão aparecer o tempo todo: classe, objeto, encapsulamento, abstração, interface, herança e polimorfismo.

Objeto e classe

A classe é o molde. O objeto é uma instância concreta desse molde.

```
class Produto {  
  constructor(  
    public nome: string,  
    public preco: number  
  ) {}  
}  
  
const cafe = new Produto("Café", 8);  
const bolo = new Produto("Bolo", 12);
```

Produto descreve o formato geral. cafe e bolo são objetos: dados reais na memória, com o mesmo formato e valores diferentes. A classe organiza o conceito e o objeto participa da execução.

Estado e comportamento

Atributos guardam dados. Métodos representam ações.

```
class ItemPedido {  
  constructor(  
    public nome: string,  
    public preco: number,  
    public quantidade: number  
  ) {}  
  
  calcularSubtotal(): number {  
    return this.preco * this.quantidade;  
  }  
}
```

nome, preco e quantidade formam o estado do objeto. calcularSubtotal() é o comportamento. O this acessa os dados daquele objeto específico, e o método fica perto dos dados que usa.

Encapsulamento

Encapsular é proteger o estado do objeto e deixar que ele mesmo controle como esse estado muda.

```
class Pedido {  
  private itens: ItemPedido[] = [];  
  
  adicionarItem(item: ItemPedido): void {  
    if (item.quantidade <= 0) {  
      throw new Error("Quantidade inválida");  
    }  
    this.itens.push(item);  
  }  
  
  calcularTotal(): number {  
    return this.itens.reduce(  

```

```

        (total, item) => total + item.calcularSubtotal(),
        0
    );
}
}

```

A lista de itens é privada e ninguém a altera por fora. O próprio Pedido decide como os itens entram, e a regra de quantidade mora no objeto responsável por ela. Quem usa o pedido não precisa conhecer os detalhes internos. Encapsulamento também protege regras de negócio.

Abstração

Abstrair é destacar o que importa e esconder os detalhes irrelevantes para quem está usando.

Imagine um código de pedido que conhece o formato do e-mail, a regra de desconto, a forma de pagamento, a geração de log e a atualização de estoque. Qualquer mudança pequena em um desses detalhes obriga a mexer no mesmo lugar.

Com abstração, o pedido conversa com conceitos como Desconto, Pagamento e Notificacao. Os detalhes concretos ficam escondidos atrás desses conceitos e podem mudar sem que o pedido perceba.

Interface

Uma interface define o que deve existir, mas não diz como será feito.

```

interface FormaPagamento {
    pagar(valor: number): void;
}

class PagamentoPix implements FormaPagamento {
    pagar(valor: number): void {
        console.log(`R$ ${valor} via PIX`);
    }
}

class PagamentoCartao implements FormaPagamento {
    pagar(valor: number): void {
        console.log(`R$ ${valor} no cartão`);
    }
}

```

As duas classes respeitam o mesmo contrato: ambas têm pagar(), mas cada uma faz isso do seu jeito. O resto do sistema pode depender de FormaPagamento e ignorar qual implementação está do outro lado.

Classe abstrata

Uma classe abstrata pode oferecer comportamento comum e deixar partes específicas para as sub-classes.

```

abstract class Notificacao {
    enviar(destino: string, mensagem: string): void {
        this.validar(destino);
        this.enviarMensagem(destino, mensagem);
    }

    private validar(destino: string): void {
        if (!destino) {
            throw new Error("Destino obrigatório");
        }
    }
}

```

```

    }

    protected abstract enviarMensagem(
        destino: string,
        mensagem: string
    ): void;
}

```

A validação é compartilhada por todas as notificações, e a forma de envio varia em cada subclasse. A diferença para a interface: a classe abstrata pode ter implementação, enquanto a interface descreve só o contrato.

Herança e composição

Herança expressa uma relação *é um*. Composição expressa uma relação *tem um*.

```

// Herança
class ClienteVip extends Cliente {}

// Composição
class Pedido {
    constructor(
        private pagamento: FormaPagamento
    ) {}
}

```

ClienteVip é um Cliente. Pedido tem uma FormaPagamento. A herança cria uma ligação forte entre as classes; a composição permite trocar partes com mais facilidade. Por isso muitos padrões preferem composição.

Uma pergunta útil antes de usar herança: isso realmente é um subtipo, ou apenas uma colaboração entre objetos?

Polimorfismo

Polimorfismo é usar o mesmo contrato e obter comportamentos diferentes.

```

function finalizarPagamento(
    pagamento: FormaPagamento,
    valor: number
): void {
    pagamento.pagar(valor);
}

finalizarPagamento(new PagamentoPix(), 100);
finalizarPagamento(new PagamentoCartao(), 100);

```

A função recebe uma FormaPagamento e não precisa saber qual implementação chegou. PIX e cartão executam comportamentos diferentes, e quem chama não precisa de um `if` para cada tipo. Essa ideia é a base de quase todos os padrões que veremos.

Acoplamento e coesão

Antes de falar de SOLID, precisamos de dois conceitos que servem para avaliar qualquer código.

Conceito	O que mede e como aparece
Acoplamento	O quanto uma parte do sistema depende de outra. Acoplamento alto: o pedido conhece detalhes do PIX, do banco, do e-mail e do logger, e uma alteração em qualquer um deles provoca mudanças em várias partes.
Coesão	O quanto as responsabilidades de uma classe pertencem ao mesmo propósito. Coesão baixa: uma única classe calcula preço, envia e-mail, salva no banco, escolhe gateway e registra log.

Tabela 1: Acoplamento e coesão.

O objetivo é acoplamento baixo e coesão alta: cada classe cuida de um assunto e depende do mínimo possível das outras.

SOLID

SOLID reúne cinco princípios para construir software mais fácil de entender, manter e modificar.

Princípio	Em português
S Single Responsibility Principle	Responsabilidade Única
O Open/Closed Principle	Aberto/Fechado
L Liskov Substitution Principle	Substituição de Liskov
I Interface Segregation Principle	Segregação de Interfaces
D Dependency Inversion Principle	Inversão de Dependência

Tabela 2: Os cinco princípios SOLID.

SOLID funciona melhor como um conjunto de perguntas do que como uma checklist. A principal delas: o que acontece com este código quando os requisitos mudarem?

S: Responsabilidade Única

Uma unidade de código deve ter um motivo bem delimitado para mudar.

```
class PedidoService {
    finalizar(pedido: Pedido): void {
        const total = this.calcularTotal(pedido);

        this.salvarNoBanco(pedido);
        this.enviarEmail(pedido.cliente);
        this.registrarLog(pedido);
    }
}
```

Essa classe tem vários motivos diferentes para mudar: a regra de cálculo, o banco, o envio de e-mail e o sistema de logs. Uma alternativa é separar essas responsabilidades:

```

class CalculadoraDePedido {
    calcular(pedido: Pedido): number {}
}

class PedidoRepository {
    salvar(pedido: Pedido): void {}
}

class Notificador {
    enviar(cliente: string): void {}
}

```

Criar uma classe para cada linha de código seria exagero. A ideia é separar o que muda por razões diferentes.

O: Aberto/Fechado

O código deve permitir extensão sem exigir alterações constantes no núcleo.

```

function calcularDesconto(tipo: string, valor: number): number {
    if (tipo === "COMUM") return 0;
    if (tipo === "ESTUDANTE") {
        return valor * 0.1;
    }
    if (tipo === "VIP") {
        return valor * 0.2;
    }
    return 0;
}

```

Cada desconto novo exige alterar essa função. Introduzindo uma abstração, novas regras entram como novas implementações:

```

interface RegraDesconto {
    calcular(valor: number): number;
}

class DescontoVip implements RegraDesconto {
    calcular(valor: number): number {
        return valor * 0.2;
    }
}

```

O princípio admite modificações no código; ele busca reduzir as alterações em regiões sensíveis. Esse problema leva naturalmente ao padrão Strategy.

L: Substituição de Liskov

Uma implementação de um contrato deve poder ser usada sem quebrar as expectativas desse contrato.

```

interface Ave {
    comer(): void;
}

interface AveQueVoa extends Ave {
    voar(): void;
}

class Pato implements AveQueVoa {

```

```

    comer(): void {}
    voar(): void {}
}

class Pinguim implements Ave {
    comer(): void {}
}

```

O problema apareceria se Ave tivesse voar() e o Pinguim fosse obrigado a implementá-lo sem conseguir voar. Um subtipo não deve quebrar as expectativas do tipo base, e um contrato não deve exigir algo que algumas implementações não conseguem cumprir. Herança mal colocada costuma revelar violações de Liskov.

I: Segregação de Interfaces

Uma classe não deveria depender de operações que não usa.

Antes

```

interface ImpressoraCompleta {
    imprimir(): void;
    escanear(): void;
    enviarFax(): void;
}

```

Depois

```

interface Impressora {
    imprimir(): void;
}

interface Scanner {
    escanear(): void;
}

interface Fax {
    enviarFax(): void;
}

```

Com a interface completa, uma impressora simples seria obrigada a implementar funções que não tem. Separando, cada classe implementa só aquilo de que precisa.

D: Inversão de Dependência

Regras importantes deveriam depender de abstrações.

Dependência direta

```

class PedidoService {
    private pagamento =
        new PagamentoPix();

    finalizar(pedido: Pedido): void {
        this.pagamento.pagar(pedido.total);
    }
}

```

Dependência de abstração

```

class PedidoService {
    constructor(
        private pagamento: FormaPagamento
    ) {}

    finalizar(pedido: Pedido): void {
        this.pagamento.pagar(pedido.total);
    }
}

```

Na primeira versão, o PedidoService escolhe, cria e usa a implementação, e fica preso ao PIX. Na segunda, ele apenas usa o contrato recebido. Isso permite passar new PagamentoPix(), new PagamentoCartao() ou new PagamentoBoleto() sem modificar o PedidoService.



DIP, IoC, DI e container

Esses quatro termos aparecem juntos e se confundem com frequência. Eles estão relacionados, mas cada um ocupa um nível diferente.

Conceito	O que é	Resumo
DIP	um princípio	dependa de abstrações
IoC	uma ideia arquitetural	a classe não controla a criação das próprias dependências
DI	uma técnica	as dependências são entregues de fora para dentro
Container	uma ferramenta	automatiza a montagem das dependências

Tabela 3: Conceitos relacionados, cada um em um nível.

Inversão de Controle (IoC)

Quando a classe cria o que usa, ela decide qual classe utilizar, quando criá-la e como construí-la:

```
class PedidoService {  
    private pagamento = new PagamentoPix();  
}
```

Quando outra parte da aplicação toma essa decisão, o controle foi invertido:

```
const pagamento = new PagamentoPix();  
const service = new PedidoService(pagamento);  
  
service.finalizar(pedido);
```

Injeção de Dependência (DI)

DI é a técnica de fornecer as dependências de fora para dentro, normalmente pelo construtor.

```
class PedidoService {  
    constructor(  
        private pagamento: FormaPagamento,  
        private notificador: Notificador  
    ) {}  
  
    finalizar(pedido: Pedido): void {  
        this.pagamento.pagar(pedido.total);  
        this.notificador.enviar(pedido.cliente);  
    }  
}
```

O `PedidoService` não cria o pagamento nem o notificador. Ele recebe os dois e usa os contratos. Trocar uma implementação passa a ser uma decisão de quem monta o serviço.

Container de DI

Quando a aplicação cresce, montar tudo à mão fica trabalhoso. Um container automatiza essa montagem:

```
container.registrar("FormaPagamento",  
    () => new PagamentoPix());
```

```

container.registrar("Notificador",
  () => new NotificadorEmail());

const service =
  container.resolver(PedidoService);

service.finalizar(pedido);

```

O container sabe como criar as dependências, quais implementações usar, como montar o grafo de objetos e quanto tempo cada instância vive. NestJS e ASP.NET usam esse conceito intensamente. DI dispensa container: na prática deste minicurso, vamos fazer a injeção manualmente.

Tempo de vida das dependências

Containers permitem definir por quanto tempo uma instância existe.

Tempo de vida	Instância	Exemplo de registro
Transient	uma nova sempre que a dependência é pedida	<code>transient(CalculadoraDeFrete)</code>
Scoped	uma por escopo, reaproveitada dentro dele	<code>scoped(CarrinhoDaRequisicao)</code>
Singleton	uma única durante toda a vida da aplicação	<code>singleton(ConfigDaAplicacao)</code>

Tabela 4: Transient, scoped e singleton. Em aplicações web, o escopo costuma ser uma requisição HTTP.

A escolha depende do estado que o objeto guarda:

```

class CalculadoraDeFrete {
  calcular(cep: string): number { return 20; }
}

class CarrinhoDaRequisicao {
  itens: ItemPedido[] = [];
}

class Configuracao {
  moeda = "BRL";
}

```

A calculadora não guarda estado entre usos, então criar uma nova a cada vez não causa problema (transient). Os serviços de uma mesma requisição compartilham o mesmo carrinho, e a próxima requisição recebe outro (scoped). Uma configuração global pode ser compartilhada pela aplicação inteira (singleton).

Cuidado

Singleton com estado mutável pode provocar problemas difíceis de rastrear, porque qualquer parte da aplicação pode alterar o que as outras estão lendo.



Design Patterns

Um design pattern é uma solução conhecida para um problema recorrente de design. Ele descreve uma maneira de organizar a colaboração entre objetos. Você não instala um pattern como uma biblioteca, e ele também não é uma regra obrigatória nem uma arquitetura completa.

Por isso, diante de um código, a pergunta que vale fazer é: “qual problema está aparecendo aqui?”. Patterns fazem mais sentido quando a mudança começa a gerar dificuldade.

Categorias clássicas

O livro da Gang of Four organiza os padrões em três grupos.

Categoria	Foco	Exemplos
Criacionais	criação de objetos	Factory Method, Abstract Factory, Builder, Prototype, Singleton
Estruturais	composição e organização de objetos	Adapter, Decorator, Facade, Composite, Proxy
Comportamentais	comunicação e distribuição de responsabilidades	Strategy, Observer, Command, State, Template Method

Tabela 5: As três categorias da Gang of Four.

Strategy

Problema: variações de um mesmo algoritmo começam a gerar condicionais.

```
interface Desconto {
    calcular(valor: number): number;
}

class DescontoEstudante
    implements Desconto {
    calcular(valor: number): number {
        return valor * 0.1;
    }
}
```

```
class PedidoService {
    constructor(
        private desconto: Desconto
    ) {}

    finalizar(valor: number): number {
        return valor -
            this.desconto.calcular(valor);
    }
}
```

Cada algoritmo vira uma estratégia com o mesmo contrato. A classe principal não conhece os detalhes, e as estratégias podem ser trocadas. Strategy conversa diretamente com o Aberto/Fechado e com a Inversão de Dependência.

Factory Method

Problema: a criação e a escolha de objetos se espalham pela aplicação.

```
class DescontoFactory {
    criar(tipoCliente: string): Desconto {
        if (tipoCliente === "ESTUDANTE") {
            return new DescontoEstudante();
        }
        if (tipoCliente === "VIP") {
            return new DescontoVip();
        }
    }
}
```

```

    return new SemDesconto();
}
}

```

Strategy responde *como calcular*. Factory responde *qual objeto criar*. A lógica de criação fica concentrada em um lugar, e o sistema deixa de espalhar new e regras de seleção.

No livro original, o Factory Method delega a criação para um método que subclasses podem sobrecrever. O exemplo acima é a forma mais simples da ideia, uma classe dedicada a criar objetos, e é a que vamos usar na prática.

Observer

Problema: uma ação principal começa a disparar várias ações secundárias. Finalizar um pedido, por exemplo, pode precisar enviar e-mail, atualizar estoque, gerar log, enviar notificação e atualizar métricas.

```

interface OuvintePedido {
    aoFinalizar(pedido: Pedido): void;
}

class PedidoService {
    constructor(
        private ouvintes: OuvintePedido[]
    ) {}

    finalizar(pedido: Pedido): void {
        // regra principal

        for (const ouvinte of this.ouvintes) {
            ouvinte.aofinalizar(pedido);
        }
    }
}

```

O serviço principal não conhece os efeitos. Novos ouvintes podem ser adicionados, e cada um tem uma responsabilidade específica. Observer é a base de listeners, subscribers e eventos.

Adapter

Problema: uma API ou biblioteca externa tem uma interface incompatível com a que o nosso sistema espera.

Nosso sistema espera

```

interface Pagamento {
    pagar(valor: number): void;
}

```

A biblioteca oferece

```

class GatewayLegado {
    executarTransacao(
        valorEmCentavos: number
    ): void {}
}

```

O adapter traduz um contrato para o outro:

```

class AdapterGatewayLegado implements Pagamento {
    constructor(
        private gateway: GatewayLegado
    ) {}

    pagar(valor: number): void {
        this.gateway.executarTransacao(valor * 100);
    }
}

```

```
}
```

É muito comum ao integrar APIs, SDKs, bibliotecas externas e sistemas legados.

Decorator

Problema: combinar comportamentos sem criar uma árvore enorme de subclasses.

O contrato

```
interface Bebida {  
    descricao(): string;  
    preco(): number;  
}
```

O objeto básico

```
class Cafe implements Bebida {  
    descricao(): string {  
        return "Café";  
    }  
    preco(): number {  
        return 8;  
    }  
}
```

O decorator implementa o mesmo contrato e envolve outro objeto:

```
class ComLeite implements Bebida {  
    constructor(private bebida: Bebida) {}  
  
    descricao(): string {  
        return this.bebida.descricao() + " + leite";  
    }  
  
    preco(): number {  
        return this.bebida.preco() + 2;  
    }  
}  
  
const bebida = new ComLeite(new Cafe());
```

O objeto original continua existindo e o decorator o envolve. Os comportamentos podem ser combinados, como `new ComChantilly(new ComLeite(new Cafe()))`, sem uma subclasse para cada combinação possível.



Como escolher um pattern

Comece pelo problema. O nome do padrão vem depois.

O que aparece no código	Pattern
Muitos if para selecionar um algoritmo	Strategy
Criação de objetos espalhada e cheia de regras	Factory
Uma ação disparando muitas ações secundárias	Observer
API externa incompatível com o seu contrato	Adapter
Muitas combinações de comportamento	Decorator

Tabela 6: Sintomas no código e o padrão que costuma resolvê-los.

Um pattern deve tornar uma mudança mais fácil de lidar.

Quando não usar

Patterns também podem gerar complexidade desnecessária. Evite aplicar um pattern só porque ele existe, principalmente quando:

- o código é simples e estável;
- existe apenas uma implementação;
- não há variação real;
- a abstração dificulta o entendimento;
- a equipe perde clareza;
- surgem mais classes sem resolver um problema real.

Bom design

Bom design reduz o custo de entender e modificar o sistema. A quantidade de patterns no código diz pouco sobre isso.



Prática: preparando o ambiente

A partir daqui vamos trabalhar no projeto da cafeteria. O código inicial funciona, mas concentra todas as decisões em uma única classe. A ideia é observar como ele fica difícil de modificar quando novas regras aparecem, e aplicar os padrões um de cada vez.

Baixando o repositório

Abra um terminal na pasta onde você guarda seus projetos e rode:

```
git clone https://github.com/AfranioCaires/cafeteria-design-patterns.git
cd cafeteria-design-patterns
npm install
npm run dev
```

O repositório também pode ser acessado pelo navegador em <https://github.com/AfranioCaires/cafeteria-design-patterns/>.

Se tudo deu certo, a saída começa assim:

```
=== PEDIDO ===
Cliente: Ana
Subtotal: R$ 24.00
Desconto: R$ 2.40
Total: R$ 21.60
Pagamento via PIX no valor de R$ 21.60
E-mail enviado para Ana
Pedido #1 registrado no histórico
Pedido finalizado com sucesso
```

Guarde essa saída. Depois de cada passo, rode `npm run dev` de novo e confira se os valores continuam iguais.

Antes de mexer no código, crie uma branch para a sua refatoração. Assim você sempre pode comparar com o código original:

```
git checkout -b minha-refatoracao
```

Conhecendo o projeto

```
src/
  database/
    BancoDeDados.ts    interface do banco
    BancoEmMemoria.ts  implementação com Map e array
  model/
    ItemPedido.ts      nome, preço e quantidade
    Pedido.ts          cliente, tipo de cliente, forma de pagamento e itens
  service/
    PedidoService.ts   onde tudo acontece
  index.ts             monta o serviço e finaliza um pedido
```

Em `src/model/Pedido.ts` estão os tipos que vamos usar bastante:

```
export type TipoCliente = "COMUM" | "ESTUDANTE" | "VIP";
export type FormaPagamento = "PIX" | "CARTAO" | "DINHEIRO";
```

O BancoEmMemoria implementa a interface BancoDeDados e guarda os pedidos e o histórico em memória. Os dados somem quando o processo termina, o que basta para a aula.

Prática: refatoração passo a passo

Cada passo resolve um problema. Ao final de cada um, o projeto continua rodando com a mesma saída. Sugestão: faça um commit por passo, para conseguir voltar se algo der errado.

Passo 0: ler o código e listar os motivos para mudar

Abra `src/service/PedidoService.ts`. O método `finalizar` faz tudo sozinho:

1. calcula o subtotal;
2. decide o percentual de desconto com um `if` por tipo de cliente;
3. imprime o resumo do pedido;
4. decide como pagar com um `if` por forma de pagamento;
5. salva no banco;
6. envia o e-mail;
7. registra no histórico.

Agora pense nas mudanças que a cafeteria pode pedir e em onde cada uma delas cairia:

Mudança pedida	Onde mexe hoje
Um desconto novo para professores	PedidoService
Uma forma de pagamento nova, como boleto	PedidoService
O cartão passa a usar uma API externa	PedidoService
Atualizar o estoque ao finalizar	PedidoService

Tabela 7: Todas as mudanças caem na mesma classe.

A mesma classe tem vários motivos diferentes para mudar, o que fere a Responsabilidade Única. Vamos atacar um motivo por vez.

Passo 1: Strategy para o desconto

Problema: variações do mesmo cálculo viraram uma sequência de `if`.

Crie a pasta `src/desconto` e, dentro dela, o arquivo `Desconto.ts` com o contrato e uma classe para cada regra:

```
export interface Desconto {  
  calcular(valor: number): number;  
}  
  
export class SemDesconto implements Desconto {  
  calcular(): number {  
    return 0;  
  }  
}
```

```

}

export class DescontoEstudante implements Desconto {
  calcular(valor: number): number {
    return valor * 0.1;
  }
}

export class DescontoVip implements Desconto {
  calcular(valor: number): number {
    return valor * 0.2;
  }
}

```

No PedidoService, importe as classes:

```

import {
  Desconto,
  DescontoEstudante,
  DescontoVip,
  SemDesconto
} from "../desconto/Desconto";

```

E troque o cálculo do percentual por este trecho:

```

let regraDesconto: Desconto = new SemDesconto();

if (pedido.tipoCliente === "ESTUDANTE") {
  regraDesconto = new DescontoEstudante();
}

if (pedido.tipoCliente === "VIP") {
  regraDesconto = new DescontoVip();
}

const desconto = Math.round(regraDesconto.calcular(subtotal) * 100) / 100;
const total = subtotal - desconto;

```

Repare que o *cálculo* saiu do serviço, mas o *if* continua lá: o serviço ainda decide *qual* desconto criar. Esse é o gancho para o próximo passo.

Rode `npm run dev`. A saída deve ser idêntica à original.

```

git add .
git commit -m "Passo 1: Strategy para o desconto"

```

Passo 2: Factory para escolher o desconto

Problema: a escolha de qual objeto criar está dentro do serviço. Strategy resolveu como calcular; agora precisamos de alguém responsável por decidir qual estratégia usar.

Crie `src/desconto/DescontoFactory.ts`:

```

import { TipoCliente } from "../model/Pedido";
import {
  Desconto,
  DescontoEstudante,
  DescontoVip,
  SemDesconto
} from "../Desconto";

export class DescontoFactory {

```

```

private readonly descontos: Record<TipoCliente, () => Desconto> = {
  COMUM: () => new SemDesconto(),
  ESTUDANTE: () => new DescontoEstudante(),
  VIP: () => new DescontoVip()
};

criar(tipoCliente: TipoCliente): Desconto {
  return this.descontos[tipoCliente]();
}
}

```

Usamos um Record no lugar da sequência de if do slide. A vantagem: se alguém criar um novo TipoCliente e esquecer de cadastrá-lo aqui, o TypeScript acusa o erro na hora de compilar.

Agora o serviço *recebe* a factory pelo construtor, aplicando a injeção de dependência. No PedidoService, troque o import das classes de desconto pelo import da factory e ajuste o construtor e o início do método:

```

import { DescontoFactory } from "../desconto/DescontoFactory";

export class PedidoService {
  constructor(
    private readonly banco: BancoDeDados,
    private readonly descontos: DescontoFactory
  ) {}

  finalizar(pedido: Pedido): PedidoRegistro {
    const subtotal = pedido.calcularSubtotal();
    const regraDesconto = this.descontos.criar(pedido.tipoCliente);
    const desconto = Math.round(regraDesconto.calcular(subtotal) * 100) / 100;
    const total = subtotal - desconto;

    // o restante do método continua igual
  }
}

```

No index.ts, passe a factory ao criar o serviço:

```

import { DescontoFactory } from "../desconto/DescontoFactory";

const pedidoService = new PedidoService(banco, new DescontoFactory());

```

Rode npm run dev e confira a saída. A partir de agora, um desconto novo não toca no PedidoService.

```

git add .
git commit -m "Passo 2: Factory de desconto e DI"

```

Passo 3: Strategy e Factory para o pagamento

Problema: é o mesmo formato do desconto, com um if por forma de pagamento.

Crie a pasta src/pagamento e o arquivo Pagamento.ts:

```

export interface Pagamento {
  pagar(valor: number): void;
}

export class PagamentoPix implements Pagamento {
  pagar(valor: number): void {
    console.log(`Pagamento via PIX no valor de R$ ${valor.toFixed(2)}`);
  }
}

export class PagamentoCartao implements Pagamento {

```

```

    pagar(valor: number): void {
      console.log(`Pagamento via cartão no valor de R$ ${valor.toFixed(2)}`);
    }
  }

export class PagamentoDinheiro implements Pagamento {
  pagar(valor: number): void {
    console.log(`Pagamento em dinheiro no valor de R$ ${valor.toFixed(2)}`);
  }
}

```

E a factory em src/pagamento/PagamentoFactory.ts:

```

import { FormaPagamento } from "../model/Pedido";
import {
  Pagamento,
  PagamentoCartao,
  PagamentoDinheiro,
  PagamentoPix
} from "../Pagamento";

export class PagamentoFactory {
  private readonly pagamentos: Record<FormaPagamento, () => Pagamento> = {
    PIX: () => new PagamentoPix(),
    CARTAO: () => new PagamentoCartao(),
    DINHEIRO: () => new PagamentoDinheiro()
  };

  criar(formaPagamento: FormaPagamento): Pagamento {
    return this.pagamentos[formaPagamento]();
  }
}

```

No serviço, os três if de pagamento viram uma linha. Veja como o PedidoService completo fica depois deste passo:

```

import { BancoDeDados, PedidoRegistro } from "../database/BancoDeDados";
import { DescontoFactory } from "../desconto/DescontoFactory";
import { Pedido } from "../model/Pedido";
import { PagamentoFactory } from "../pagamento/PagamentoFactory";

export class PedidoService {
  constructor(
    private readonly banco: BancoDeDados,
    private readonly descontos: DescontoFactory,
    private readonly pagamentos: PagamentoFactory
  ) {}

  finalizar(pedido: Pedido): PedidoRegistro {
    const subtotal = pedido.calcularSubtotal();
    const regraDesconto = this.descontos.criar(pedido.tipoCliente);
    const desconto = Math.round(regraDesconto.calcular(subtotal) * 100) / 100;
    const total = subtotal - desconto;

    console.log("=== PEDIDO ===");
    console.log(`Cliente: ${pedido.cliente}`);
    console.log(`Subtotal: R$ ${subtotal.toFixed(2)}`);
    console.log(`Desconto: R$ ${desconto.toFixed(2)}`);
    console.log(`Total: R$ ${total.toFixed(2)}`);

    this.pagamentos.criar(pedido.formaPagamento).pagar(total);
  }
}

```

```

const registro = this.banco.salvarPedido({
  cliente: pedido.cliente,
  tipoCliente: pedido.tipoCliente,
  formaPagamento: pedido.formaPagamento,
  subtotal,
  desconto,
  total,
  status: "FINALIZADO"
});

console.log(`E-mail enviado para ${pedido.cliente}`);

this.banco.registrarHistorico(registro.id, "Pedido finalizado");
console.log(`Pedido #${registro.id} registrado no histórico`);
console.log("Pedido finalizado com sucesso");

return registro;
}
}

```

No `index.ts`, importe a `PagamentoFactory` e passe `new PagamentoFactory()` como terceiro argumento do serviço. Rode `npm run dev`: a saída continua a mesma. Uma forma de pagamento nova agora pede só uma classe e uma linha na `factory`.

```

git add .
git commit -m "Passo 3: Strategy e Factory de pagamento"

```

Passo 4: Adapter para o cartão

Requisito novo: o cartão passa a ser cobrado por uma biblioteca externa. Ela tem outro contrato: o método se chama `executarTransacao` e recebe o valor em centavos.

Crie `src/pagamento/GatewayLegado.ts` para simular essa biblioteca. Em um projeto real, esse código viria de um pacote e você não poderia alterá-lo:

```

// Simula uma biblioteca externa de cartão, com um contrato diferente do nosso.
export class GatewayLegado {
  executarTransacao(valorEmCentavos: number): void {
    console.log(
      `[GatewayLegado] Transação de cartão aprovada: ${valorEmCentavos} centavos`
    );
  }
}

```

Crie o adapter em `src/pagamento/AdapterGatewayLegado.ts`. Ele implementa o nosso contrato e traduz a chamada para o contrato da biblioteca:

```

import { GatewayLegado } from "../GatewayLegado";
import { Pagamento } from "../Pagamento";

export class AdapterGatewayLegado implements Pagamento {
  constructor(private readonly gateway: GatewayLegado) {}

  pagar(valor: number): void {
    this.gateway.executarTransacao(Math.round(valor * 100));
  }
}

```

O `Math.round` evita erros de ponto flutuante: `16.1 * 100` dá `1610.0000000000002` em JavaScript. Na `PagamentoFactory`, troque a linha do cartão e ajuste os imports:

```
CARTAO: () => new AdapterGatewayLegado(new GatewayLegado()),
```

Apague a classe `PagamentoCartao` de `Pagamento.ts`, que deixou de ser usada. Para ver o cartão funcionando, adicione no `index.ts` um pedido pago com cartão:

```
pedidoService.finalizar(  
  new Pedido("Bruno", "VIP", "CARTAO", [  
    new ItemPedido("Café com leite", 10, 2)  
  ])  
);
```

Rode `npm run dev` e procure a linha `[GatewayLegado] Transação de cartão aprovada: 1600 centavos`.

Para discutir com a turma

O requisito mudou e o `PedidoService` não foi tocado. Esse é o ganho acumulado dos passos 2 e 3: a decisão de como pagar mora na `factory`, e o `adapter` isolou o contrato estranho da biblioteca.

```
git add .  
git commit -m "Passo 4: Adapter para o gateway de cartão"
```

Passo 5: Observer para as ações depois de finalizar

Problema: finalizar o pedido dispara várias ações secundárias (resumo, e-mail e histórico), e a lista tende a crescer com estoque, métricas e outras.

Crie a pasta `src/ouvintes` e o contrato em `OuvintePedido.ts`:

```
import { PedidoRegistro } from "../database/BancoDeDados";  
  
export interface OuvintePedido {  
  aoFinalizar(pedido: PedidoRegistro): void;  
}
```

O ouvinte recebe o `PedidoRegistro`, que já foi salvo e tem `id`, `desconto` e `total`. É disso que as ações precisam.

Agora crie um ouvinte para cada ação que hoje está no serviço, todos na pasta `src/ouvintes`. O primeiro é o `ResumoNoConsole.ts`:

```
import { PedidoRegistro } from "../database/BancoDeDados";  
import { OuvintePedido } from "../OuvintePedido";  
  
export class ResumoNoConsole implements OuvintePedido {  
  aoFinalizar(pedido: PedidoRegistro): void {  
    console.log(`=== PEDIDO #${pedido.id} ===`);  
    console.log(`Cliente: ${pedido.cliente}`);  
    console.log(`Subtotal: R$ ${pedido.subtotal.toFixed(2)}`);  
    console.log(`Desconto: R$ ${pedido.desconto.toFixed(2)}`);  
    console.log(`Total: R$ ${pedido.total.toFixed(2)}`);  
  }  
}
```

Depois `src/ouvintes/NotificadorEmail.ts`:

```
import { PedidoRegistro } from "../database/BancoDeDados";  
import { OuvintePedido } from "../OuvintePedido";  
  
export class NotificadorEmail implements OuvintePedido {
```

```

    aoFinalizar(pedido: PedidoRegistro): void {
      console.log(`E-mail enviado para ${pedido.cliente}`);
    }
  }
}

```

É src/ouvintes/HistoricoPedido.ts, que precisa do banco e o recebe pelo construtor:

```

import { BancoDeDados, PedidoRegistro } from "../database/BancoDeDados";
import { OuvintePedido } from "../OuvintePedido";

export class HistoricoPedido implements OuvintePedido {
  constructor(private readonly banco: BancoDeDados) {}

  aoFinalizar(pedido: PedidoRegistro): void {
    this.banco.registrarHistorico(pedido.id, "Pedido finalizado");
    console.log(`Pedido #${pedido.id} registrado no histórico`);
  }
}

```

No serviço, apague todos os console.log e a chamada a registrarHistorico. Receba a lista de ouvintes pelo construtor e avise cada um no final. O PedidoService completo fica assim:

```

import { BancoDeDados, PedidoRegistro } from "../database/BancoDeDados";
import { DescontoFactory } from "../desconto/DescontoFactory";
import { Pedido } from "../model/Pedido";
import { OuvintePedido } from "../ouvintes/OuvintePedido";
import { PagamentoFactory } from "../pagamento/PagamentoFactory";

export class PedidoService {
  constructor(
    private readonly banco: BancoDeDados,
    private readonly descontos: DescontoFactory,
    private readonly pagamentos: PagamentoFactory,
    private readonly ouvintes: OuvintePedido[] = []
  ) {}

  finalizar(pedido: Pedido): PedidoRegistro {
    const subtotal = pedido.calcularSubtotal();
    const regraDesconto = this.descontos.criar(pedido.tipoCliente);
    const desconto = Math.round(regraDesconto.calcular(subtotal) * 100) / 100;
    const total = subtotal - desconto;

    this.pagamentos.criar(pedido.formaPagamento).pagar(total);

    const registro = this.banco.salvarPedido({
      cliente: pedido.cliente,
      tipoCliente: pedido.tipoCliente,
      formaPagamento: pedido.formaPagamento,
      subtotal,
      desconto,
      total,
      status: "FINALIZADO"
    });

    for (const ouvinte of this.ouvintes) {
      ouvinte.aoFinalizar(registro);
    }

    return registro;
  }
}

```

Rode npm run dev. Os valores são os mesmos, mas a ordem das linhas muda: o pagamento aparece

antes do resumo, porque o resumo virou um ouvinte que roda depois de salvar. A mensagem “Pedido finalizado com sucesso” também sai, já que era só mais um log dentro do serviço.

```
git add .
git commit -m "Passo 5: Observer para as acoes pos-finalizacao"
```

Passo 6: montar tudo no index.ts

O index.ts passa a ser o único lugar que conhece as classes concretas. Ele monta os objetos e entrega ao serviço, sem container:

```
import { BancoEmMemoria } from "../database/BancoEmMemoria";
import { DescontoFactory } from "../desconto/DescontoFactory";
import { ItemPedido } from "../model/ItemPedido";
import { Pedido } from "../model/Pedido";
import { HistoricoPedido } from "../ouvintes/HistoricoPedido";
import { NotificadorEmail } from "../ouvintes/NotificadorEmail";
import { ResumoNoConsole } from "../ouvintes/ResumoNoConsole";
import { PagamentoFactory } from "../pagamento/PagamentoFactory";
import { PedidoService } from "../service/PedidoService";

// Composição manual das dependências (DI sem container).
const banco = new BancoEmMemoria();
const pedidoService = new PedidoService(
  banco,
  new DescontoFactory(),
  new PagamentoFactory(),
  [new ResumoNoConsole(), new NotificadorEmail(), new HistoricoPedido(banco)]
);

const pedidos = [
  new Pedido("Ana", "ESTUDANTE", "PIX", [
    new ItemPedido("Cappuccino", 12, 1),
    new ItemPedido("Pão de queijo", 6, 2)
  ]),
  new Pedido("Bruno", "VIP", "CARTAO", [
    new ItemPedido("Café com leite", 10, 2)
  ]),
  new Pedido("Carla", "COMUM", "DINHEIRO", [new ItemPedido("Café", 8, 1)])
];

for (const pedido of pedidos) {
  pedidoService.finalizar(pedido);
  console.log();
}

console.log("=== BANCO EM MEMÓRIA ===");
console.log(banco.listarPedidos());
console.log(banco.listarHistorico());
```

A saída esperada para os três pedidos:

```
Pagamento via PIX no valor de R$ 21.60
=== PEDIDO #1 ===
Cliente: Ana
Subtotal: R$ 24.00
Desconto: R$ 2.40
Total: R$ 21.60
E-mail enviado para Ana
Pedido #1 registrado no histórico
```

```
[GatewayLegado] Transação de cartão aprovada: 1600 centavos
=== PEDIDO #2 ===
Cliente: Bruno
Subtotal: R$ 20.00
Desconto: R$ 4.00
Total: R$ 16.00
E-mail enviado para Bruno
Pedido #2 registrado no histórico
```

```
Pagamento em dinheiro no valor de R$ 8.00
=== PEDIDO #3 ===
Cliente: Carla
Subtotal: R$ 8.00
Desconto: R$ 0.00
Total: R$ 8.00
E-mail enviado para Carla
Pedido #3 registrado no histórico
```

```
git add .
git commit -m "Passo 6: composicao das dependencias no index"
```

Como o projeto ficou



Figura 1: O serviço depende só de contratos e factories; o `index.ts` escolhe as implementações.

Problema no código inicial	Solução aplicada
if por tipo de cliente para o desconto	Strategy Desconto + Factory
if por forma de pagamento	Strategy Pagamento + Factory
Cartão via biblioteca com contrato diferente	Adapter AdapterGatewayLegado
E-mail, resumo e histórico dentro do serviço	Observer OuvintePedido
PedidoService escolhendo e criando tudo	DI pelo construtor

Tabela 8: Resumo da refatoração.

O `PedidoService` agora só coordena o fluxo. As decisões ficaram em classes que têm um único motivo para mudar.

Exercícios

Faça cada exercício sem editar o `PedidoService`. Se precisar abri-lo, pare e pense em qual passo ficou incompleto.

Exercício	Onde mexer
Desconto de 15% para clientes PROFESSOR	<code>TipoCliente</code> , nova classe de Desconto e <code>DescontoFactory</code>
Pagamento por BOLETO	<code>FormaPagamento</code> , nova classe de Pagamento e <code>PagamentoFactory</code>
Atualizar o estoque ao finalizar	novo <code>OuvintePedido</code> , registrado no <code>index.ts</code>
Registrar métricas de vendas por forma de pagamento	novo <code>OuvintePedido</code> , registrado no <code>index.ts</code>

Tabela 9: Exercícios para fixar os padrões.

No primeiro exercício, repare no que acontece quando você adiciona "PROFESSOR" ao `TipoCliente` e ainda não mexeu na factory: o TypeScript aponta o erro no `Record`. Esse é o motivo de termos trocado a sequência de `if` por um mapa.

Para continuar estudando

Obra	Por que ler
Design Patterns: Elements of Reusable Object-Oriented Software	Gamma, Helm, Johnson e Vlissides (Gang of Four). A referência clássica que formalizou os 23 padrões.
Head First Design Patterns	Freeman e Robson. Abordagem visual e didática.
Agile Software Development: Principles, Patterns, and Practices	Robert C. Martin. SOLID, orientação a objetos, design, dependências e padrões.
Refactoring.Guru	Material visual sobre Design Patterns, SOLID, refatoração e UML: https://refactoring.guru .

Tabela 10: Referências.

Fechamento

Design Patterns são nomes dados a soluções que aparecem repetidamente quando o software começa a mudar. Mais útil do que decorar a lista é reconhecer o problema que cada um resolve.

- Orientação a objetos fornece o vocabulário.
- SOLID ajuda a reconhecer problemas de design.
- DIP reduz a dependência de implementações concretas.
- IoC muda quem controla a criação das dependências.

- DI fornece essas dependências de fora para dentro.
- Patterns oferecem estruturas conhecidas para problemas conhecidos.

Antes de escolher um pattern, entenda qual problema você está tentando resolver.
